

Master Data & IA

Academic year 2026/2027

Assignment Report : Interpolation – **Lagrange** & **Newton**

Comparison between the main function and the interpolation
polynomial

Module : Mathematics for AI

Prepared by : HAMZA EL HAKKOUNI

1. Lagrange Interpolation :

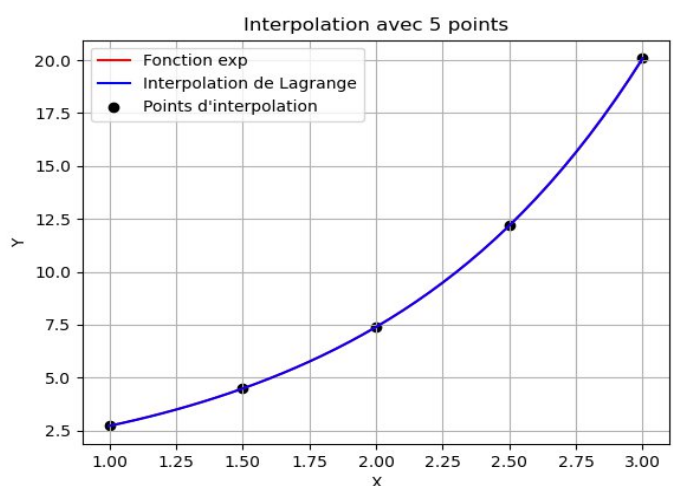
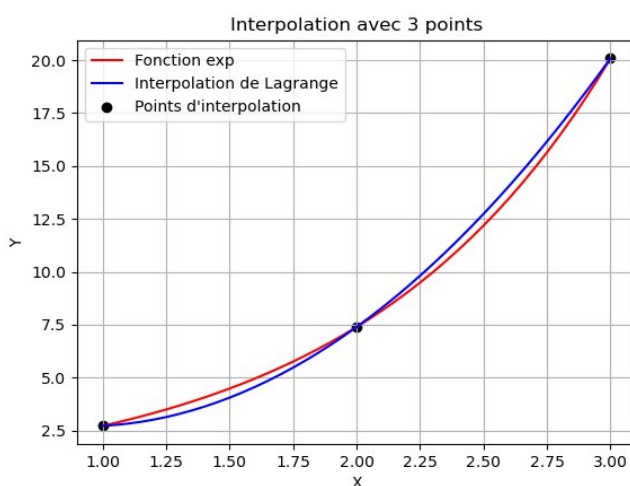
$$f(x) = f(x_0) \frac{(x-x_1)(x-x_2)\cdots(x-x_n)}{(x_0-x_1)(x_0-x_2)\cdots(x_0-x_n)} + f(x_1) \frac{(x-x_0)(x-x_2)\cdots(x-x_n)}{(x_1-x_0)(x_1-x_2)\cdots(x_1-x_n)} + \cdots + f(x_n) \frac{(x-x_0)(x-x_1)\cdots(x-x_{n-1})}{(x_n-x_0)(x_n-x_1)\cdots(x_n-x_{n-1})}$$

This is called Lagrange's interpolation formula and can be used both equal and unequal intervals .

Python Code :

We applied Lagrange interpolation to the function :

$f(x) = \exp(x)$ using 3 and then 5 points. As shown in the graphs, the polynomial **$P(x)$** gets closer to the real curve when we increase the number of points. This shows that accuracy improves with more interpolation nodes.



```

import math

def CalculateLi(x , i , x_points):

    result = 1

    n = len(x_points)

    for j in range(n) :

        if(j != i):

            result = result * ((x - x_points[j]) / (x_points[i] -
x_points[j]))

    return result

def Lagrange(x , x_points , y_points):

    result = 0

    n = len(x_points)

    for i in range(n):

        result = result + (y_points[i] * CalculateLi(x , i , x_points))

    return result

# maintenant on va tester avec 3 points :

# On choisit la fonction exp :

# on choisit x = 1.5

# la vraie valeur

print("Exp(1.5) = " , math.exp(1.5))

# par lagrange

x_points = [1 , 2 , 3]

y_points = [ math.exp(1) , math.exp(2) , math.exp(3)]

r = Lagrange( 1.5 , x_points , y_points)

print("P(1.5) = " , r)

# maintenant on augmente le nombre du points :

# la vraie valeur

print("Exp(1.5) = " , math.exp(1.5))

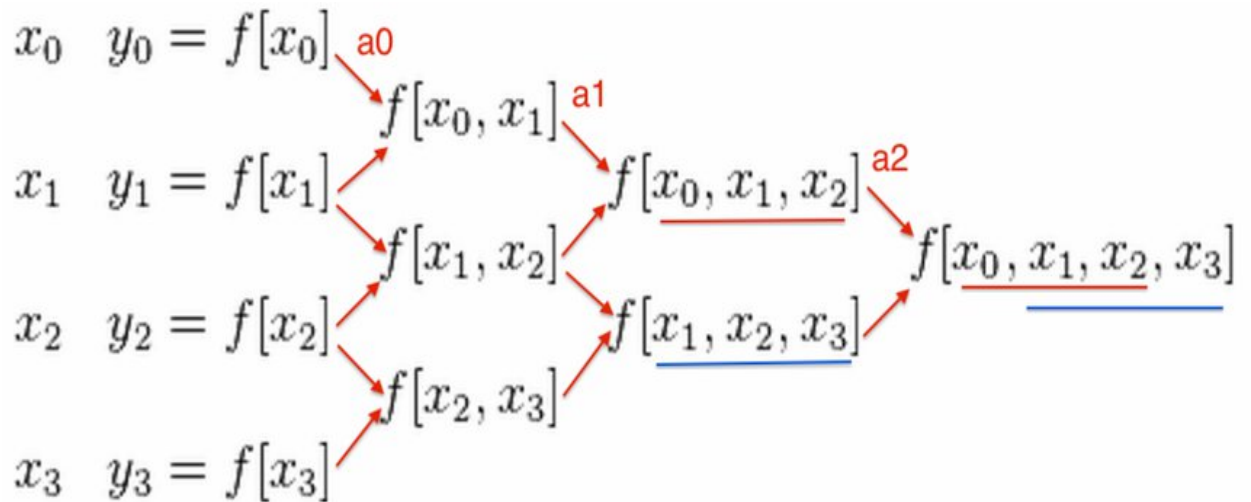
# par lagrange

x_points = [1 , 1.5 , 2 , 2.5 , 3]

```

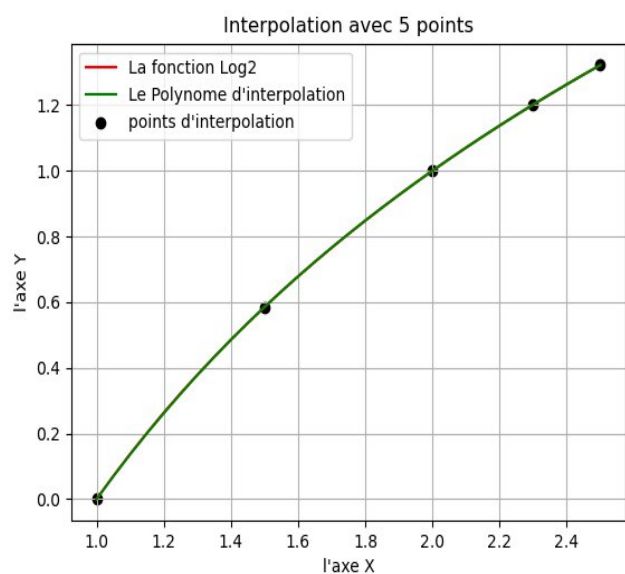
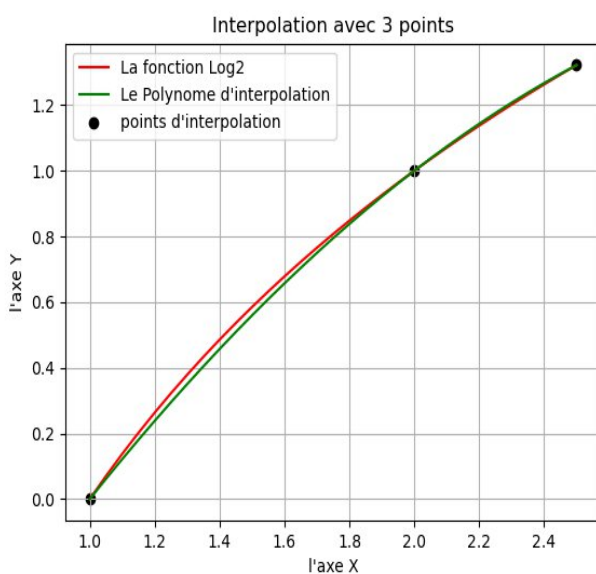
```
y_points = [ math.exp(1) , math.exp(1.5) , math.exp(2) , math.exp(2.5) ,  
math.exp(3)]  
  
r = Lagrange( 1.5 , x_points , y_points)  
  
print("P(1.5) = " , r)  
  
# donc si on augmente le nombre des points la précision est augmente
```

2. Newton Interpolation :



Python Code :

We used Newton's method to test the **ln(x)** function. We can see that adding more points between 1 and 2.5 makes the polynomial curve more accurate.



```

import math

def CalculateCoefficient(i , x_points , x):
    produit = 1
    for j in range(i):
        produit = produit * ( x - x_points[j])
    return produit

def DevidedDifference(x_points , y_points):
    n = len(x_points)
    matrice = [[0.0 for _ in range(n)] for _ in range(n)]
    for j in range(n):
        for k in range(n):
            matrice[j][k] = 0
    for j in range(n):
        matrice[j][0] = y_points[j]
    for j in range(1 , n):
        for k in range(n - j ):
            matrice[k][j] = (matrice[k+1][j-1] - matrice[k][j-1] ) /
(x_points[j + k] - x_points[k])
    return matrice

def ConstruirePolynome(x_points , y_points , x):
    n = len(x_points)
    matrice = DevidedDifference(x_points , y_points)
    result = matrice[0][0]
    for j in range(1, n):
        result += matrice[0][j] * CalculateCoefficient(j, x_points, x)
    return result

# Exemple de test avec la fonction ln(x) et 3 points
#On Va calculer ln(1.5) et P(1.5)
print("ln(1.5) = ", math.log2(1.5))

```

```

#maintenant calculons le polynome a partir des points ( 1 , ln(1)) , ( 2 ,
ln(2)) , (2.5 , ln(2.5))

x_points = [ 1 , 2 , 2.5]

y_points = [math.log2(1) , math.log2(2) , math.log2(2.5)]

print("P(1.5) = " , ConstruirePolynome(x_points , y_points , 1.5))

#maintenat en augmente le nombre de points dans l'intervalle [1 , 2.5]

# (1 , ln(1)) , (1.5 , ln(1.5)) , (2 , ln(2)) , (2.3 , ln(2.3)) , (2.5 ,
ln(2.5))

print("Apres l'ajout des points :")

print("ln(1.5) = " , math.log2(1.5))

x_points = [ 1 , 1.5 , 2 , 2.3 , 2.5]

y_points = [math.log2(1) , math.log2(1.5) , math.log2(2) , math.log2(2.3),
math.log2(2.5)]

print("P(1.5) = " , ConstruirePolynome(x_points , y_points , 1.5))

```

3. Conclusion :

In conclusion, this practical study allowed us to compare the Lagrange and Newton interpolation methods :

- **Accuracy:** For both methods, adding more interpolation points within the interval reduces the error and brings the polynomial closer to the actual function.
- **Equivalence:** Both algorithms produce the exact same final result, but Newton's method is computationally more convenient when adding new points, as it does not require recalculating everything from scratch.